Baseline Reproduction

Team 27

Date: October 19, 2026

Subject: Baseline Reproduction - Clustering NBA Player-Seasons by Playstyle

Repository Submission

- Repository URL: https://github.com/AllenBlesson/NBA_ML
- Release Tag: v1.0-baseline

Reproducible Environment and Repository Setup

The codebase for this assignment is tagged as v1.0-baseline in the main repository. Reproducibility is guaranteed via a requirements.txt file (using Python 3.14.) located in the root directory. Users can recreate the environment using pip install -r requirements.txt. All stochastic processes (model initialization and the stability checks) are locked using seed = 0, and the train/validation/test split is fixed by season.

- Run order from a fresh clone:
    1. pip install -r requirements.txt
    2. Download the dataset (CC0, Kaggle: rodneycarroll78/nba-stats-1980-2024) into data/raw/nba_stats/
    3. python src/pipeline.py
    4. python src/baseline.py

1. Summary

This milestone establishes a working data pipeline and a set of baselines for clustering NBA player-seasons into play-style groups. Later models (the complementarity rule and the team-outcome check) must beat these baselines to be worth using. All results use the season-based split defined in Milestone 2.

2. Working Data Pipeline

Run python src/pipeline.py from the repo root. It reads data/raw/nba_stats/Advanced.csv and writes data/processed/cleaned_nba.csv and data/processed/log.json.

Step	Rule	Rows after step
Raw file	All seasons in the Kaggle file	31870
Types and duplicates	Convert to numeric; drop duplicate rows	31870
Season filter	Keep 2016-2024 (labelled by ending year; 2016 means 2015-16)	6126
Missing values	Drop rows with missing selected features	6081
Traded players	Keep the combined TOT row; drop per-team rows	4789
Minutes filter	Keep 500+ minutes	3222
Split	Train 2016-2021, validation 2022-2023, test 2024	Train: 2120, Val: 742, Test:360

Features: usg_percent, ts_percent, x3p_ar, ast_percent, trb_percent. The scaler is fit on training seasons only.

3. Benchmarks

Role	Method	Why it is here
Naive benchmark	Random cluster labels with the same k, a single random draw (seed 0)	Shows what "no structure" scores
Reference	Traditional position labels	The grouping already in use; clusters should add something beyond it

Course-level baseline	k-means on standardized features, k chosen on validation (k = 2 to 8)	Standard method from the course
Candidate	Gaussian mixture, same k as k-means	Allows overlapping, elliptical groups and gives membership probabilities

4. Baseline Table

model	k	silhouette_val	silhouette_test	davies_bouldin_test
Naive: random labels	2	0.0014	-0.0017	22.3586
Reference: position labels	15	-0.226	-0.3513	3.4313
Course-level baseline: k-means	2	0.2991	0.3018	1.3604
Candidate: Gaussian mixture	2	0.2949	0.3005	1.2544

5. Discrepancy Analysis

Item	Expected	Observed	Likely reason
Clusters vs. position labels	Clusters score at least as high	Clusters score far higher. Silhouette is 0.2991 (val) and 0.3018 (test) for k-means, versus -0.226 and -0.3513 for position labels. Davies-Bouldin on test is 1.36 versus 3.43.	Position labels (15 categories, including hybrids) overlap heavily in usage, shooting, passing and rebounding space, so they are not compact groups in these five features. The comparison is not like-for-like: k = 15 versus k = 2, and silhouette favours small k.
k-means vs. GMM	Similar groupings	ARI = 0.7998	ARI of about 0.80 on the test means that the two methods agree on most players. They differ mostly near the boundary between the two groups
Validation vs. test	Similar scores	k-means is 0.2991 (val) vs 0.3018 (test); GMM is 0.2949 vs 0.3005. The gap is under 0.01, with test slightly higher.	Test is one season, so smaller and noisier
Choice of k	Interpretable clusters	Best k = 2	Silhouette tends to favour small k; compare with k_selection.csv
3-point attempt rate	Roughly bell-shaped	Spike at 0 (players who never shoot threes)	Real feature of the data; non-Gaussian, which may hurt GMM
Era drift	Stable across seasons	3PA rate rose from 0.282 (2016) to 0.402 (2024)	One global scaler keeps the drift; try within-season standardization
Stability	Same clusters across seeds	Mean ARI = k-means:1.0 GMM:0.65	K-means gives the same groups every time, and GMM changes depending on the starting point so it's less reliable

6. Start / Stop / Continue

- **Start:** writing the complementarity rule before looking at results; running the team-outcome check on held-out seasons; testing within-season standardization as a sensitivity check.
- **Stop:** judging cluster quality by silhouette alone; relying on a single test season for conclusions.
- **Continue:** season-based splits with the scaler fit on train only; fixed seeds and pinned packages; logging pipeline counts to log.json.